FROM TOOLS TO TEAMMATES

THE RISE OF AI AGENTS IN SOFTWARE DEVELOPMENT

By Ashadeepa Debnath

2026

JS COMMUNITY

Why Today's Tools Aren't Enough

THE SHIFT

- Modern software is a living city of services, APIs, pipelines, and teams.
- The challenge is not only building components but coordinating them reliably at scale.

TOOLS ARE REACTIVE

- Autocomplete, linters, and copilots help developers move faster — but they wait for prompts.
- They reduce effort without owning goals, context, or outcomes.

THE HUMAN BOTTLENECK

- When every decision still routes through a person, delivery slows.
- The system can be powerful, but the orchestration layer remains manual and fragile.

JS COMMUNITY

Systems are powerful, but orchestration is still manual and fragile

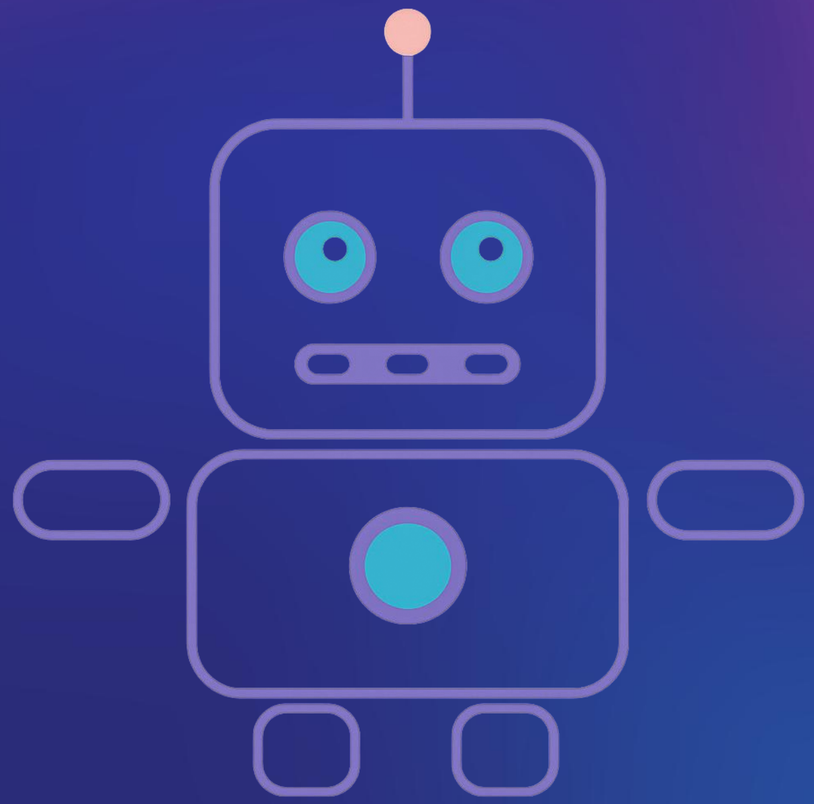
Who Am I

- **AI-enabled architect** and **developer at** ❤️ with a mission to build scalable, secure, future-ready platforms across frontend, backend, and cloud.
- A silent force of innovation with **3 US-filed & 2-granted patents**.
- Bringing **13 years of IT experience** and the precision of a top-tier operative to every mission.

Technologist, Passionate, Creative!

JS COMMUNITY

The Shift Nobody Noticed



Era 1 — Autocomplete (2022)

Copilot, code suggestions. Stateless. You invoke, it predicts.
Same mental model as a very smart search.
No goals, no context, no outcomes.

Era 2 — Conversation (2023)

ChatGPT, Claude. Turn-based. You ask, it answers.
You are still the executor — you copy-paste, you run, you decide.

Era 3 — Agency (2024–now)

The model reads your repo, creates a branch, writes tests, runs them, reads the failure, tries again, and opens a PR. You review the PR. You are no longer the executor.

JS COMMUNITY

LLMs VS Coding Agents

LLM

User prompt

↓

LLM
Predicts next token

↓

Text response

Characteristics

- Single inference pass
- No memory across turns
- No tool access
- Stateless
- Answers from training data
- Best for: Q&A, drafting, explain

Coding Agent

User task

↓

LLM (reasoning)
Plans + generates actions

↓

Read file | Run code | Write file

↻ observe → decide → act loop

↓

Working code / artifact

Characteristics

- Multi-step agentic loop
- Uses tools (files, shell, APIs)
- Maintains state across steps

JS COMMUNITY

LLM : Single forward pass. No memory, no tools, no state. **Coding Agents** : Looping agent with tools, memory, and action feedback!

Anatomy of an Agent

Perception → Plan → Act → Remember

Perceive

Receive input — a user message, a tool result, a file read. Everything the agent knows about the current task lives in the context window.

1

Plan

Decide what to do next via chain-of-thought reasoning. Modern LLMs do this implicitly. Some frameworks make it explicit with a separate planner model.

2

Act

Call a tool, write a file, send a message, spawn a sub-agent. This is where real-world effects happen. Tool use is the upgrade from chatbot to agent.

3

Remember

Store state for the next iteration — in-context, episodic (vector store), or procedural (structured state object). Context windows are the new heap.

4

loop

Context Engineering

What is Context?

Everything in the model's window: system prompt, conversation history, tool results, injected docs. Quality > quantity.

Compaction

Summarize old turns, prune stale data, keep recent events verbatim. Prevents context overflow and cost blowout.

Context Rot

Gradual degradation from irrelevant history. Symptoms: hallucinations, repetition, missed instructions.

Context Window

STAY ZONE

Recent events, active task, current file

Always keep verbatim

COMPRESS

Older turns, completed sub-tasks

Summarize before overflow

PRUNE

Stale context, resolved errors, old docs

Remove to fight context rot

JS COMMUNITY

Context Engineering

Context Window

200,000 Token Limit

System Prompt
Gives the LLM an identity (i.e. "you are Jarvis")

Current User Prompt
i.e. Fix my failing unit tests

Instructions
i.e. CLAUDE.md, .github/copilot-instructions.md

Available Tools
i.e. Bash, github_create_pr, get_jira_item, etc.

Sub-Agent Names
i.e. .claude/agents, .github/agents

Skill Names
i.e. .claude/skills, .github/skills

Plans and TODO lists
Strategic planning & execution roadmap

Previous Tool Responses
i.e. code read, PRs created, test results

Chat History
Conversation context & decisions made

↓ Free Space ↓

Dynamically Loaded

When Needed

Agents

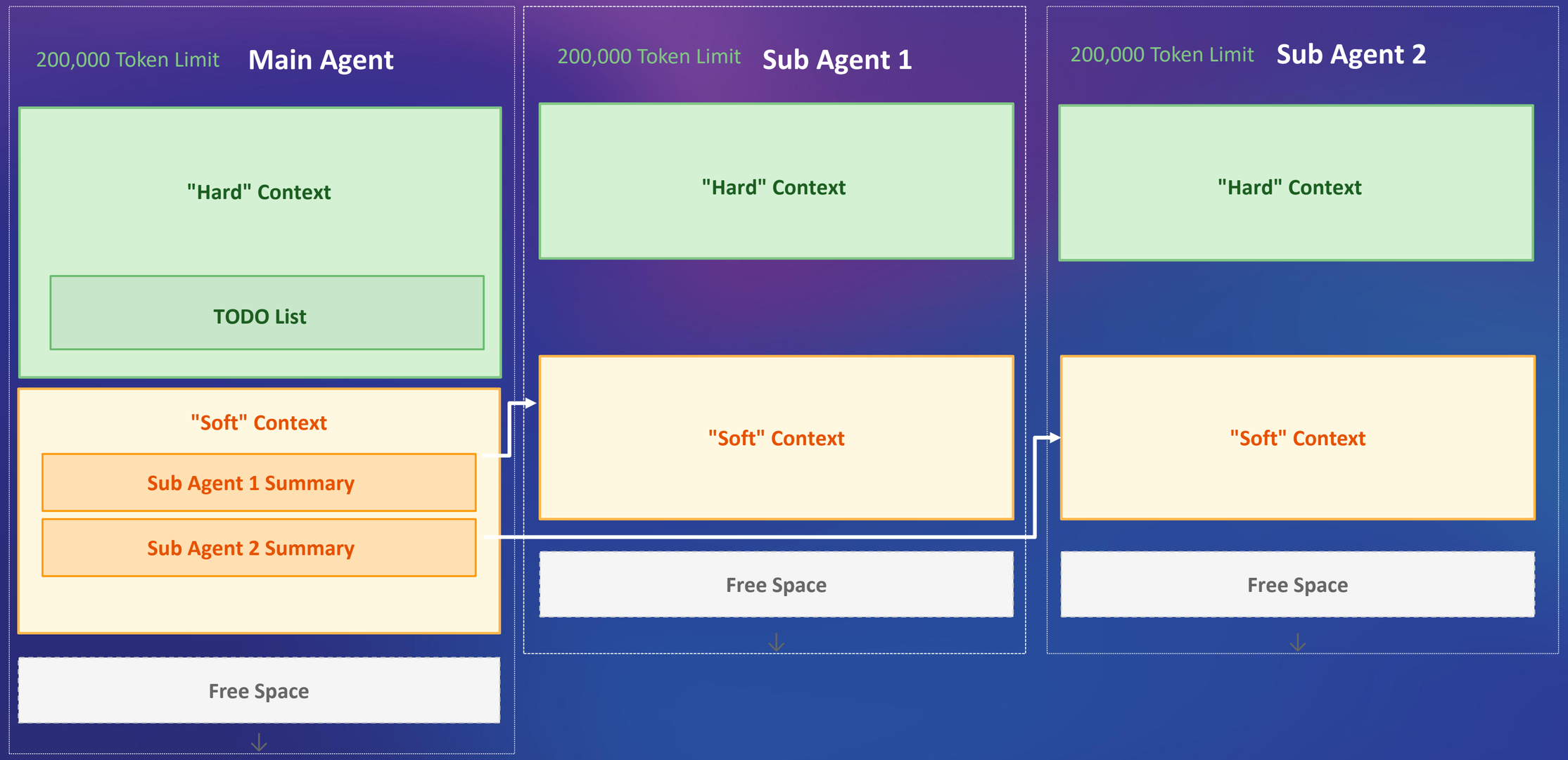
- Agent 1
- Agent 2
- . . .

Skills

- Skill 1
- Skill 2
- . . .

Context engineering is the highest-leverage skill in agentic AI development. Master the window, master the agent!

Sub Agents



Agents can have unique roles with unique perspectives!

Agent Skills (agentskills.io)

Agent

200,000 Token Limit

"Hard" Context

Available Skills:
- xlsx
- ppt
- webapp-testing

SKILL.md
scripts/
references/
assets/

"Soft" Context

Free Space

↓

Dynamically Loaded

When Needed

"xlsx" Skill

"ppt" Skill

"webapp-testing" Skill

"c4-diagrams" Skill

Agents only load the skills they need, maximizing available context space!

Anatomy of a Skill

```
my-skill/
  ├── SKILL.md
  |   ├── YAML frontmatter
  |   |   (name, description)
  |   └── Markdown instructions
  ├── scripts/
  |   └── Reusable code
  ├── references/
  |   └── Docs loaded on demand
  └── assets/
      └── Templates, fonts, icons
```

1 Metadata

Name + description always in context. ~100 words. This is the trigger.

2 SKILL.md Body

Full instructions loaded when the skill triggers. Keep under 500 lines.

3 Bundled Resources

Scripts, references, assets loaded as needed. Unlimited size.

A skill is a mini-playbook that tells your agent exactly how to think and act!

JS COMMUNITY

My Creation : The Accessibility Guardian

The Agent: A11y-Healer
File: plugin/a11y-healer/agents/a11y-healer.md

An interactive agent that operates in action mode — it doesn't just suggest changes; it applies them after user confirmation. Workflow:

Explain what it will change and why
Wait for approval
Execute the edit via tools
Verify the change was applied
Run tests to confirm the fix works
Standards enforced: WCAG 2.2 AA, ARIA 1.2, Section 508, ADA.

The agent has an embedded WCAG quick reference covering keyboard patterns (Tab/Shift Tab, arrow keys, Esc, focus order, skip links) and ARIA semantics.

A11y-Healer Agent
├── Skill: accessibility-rule-detection — WCAG 2.2 violation detection across all POUR principles
│ ├── Detects: missing alt text, low contrast, missing labels
│ ├── Detects: keyboard traps, focus issues
│ └── Detects: improper heading hierarchy, misuse of roles
│
├── Skill: aria-fix-generation — Accessible names, ARIA roles/states, error associations, live regions
│ ├── Adds aria-label / aria-labelledby
│ ├── Fixes role attributes and states
│ └── Associates form inputs with error messages
│
├── Skill: semantic-html-correction — Native HTML elements, heading hierarchy, landmarks, contrast, focus indicators
│ ├── Replaces div/span with semantic elements (button, nav, header)
│ ├── Fixes heading order (h1 → h6)
│ └── Ensures proper landmarks and focus visibility
│
└── Skill: accessibility-testing — axe-core + Playwright/Jest/Cypress, CLI tools, CI/CD integration, manual testing
├── Runs automated accessibility scans
├── Generates accessibility reports
└── Validates fixes before merge in CI/CD

Tweak the Prompt, Not the Code

Your prompt IS your program. Quality in → quality out.

❌ BAD

Bad Prompt

"Write code for my app"

No context · No role · No constraints · No goal

LLM

processes context

Bad Output

"Generic boilerplate,
wrong framework, no tests"

Useless · Needs complete rewrite

✅ GOOD

Good Prompt

**"You are a senior TS dev.
Add auth to this Express route..."**

Role · Context · Constraints · Goal

LLM

same model, same size

Great Output

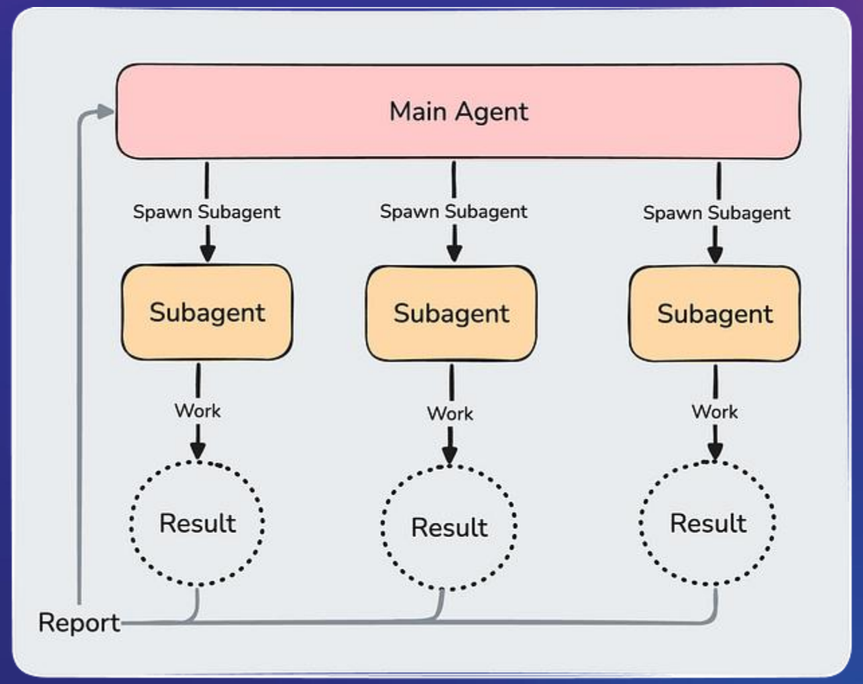
"JWT middleware, error handling,
tests, typed, production-ready"

Ready to ship · Zero rewrite needed

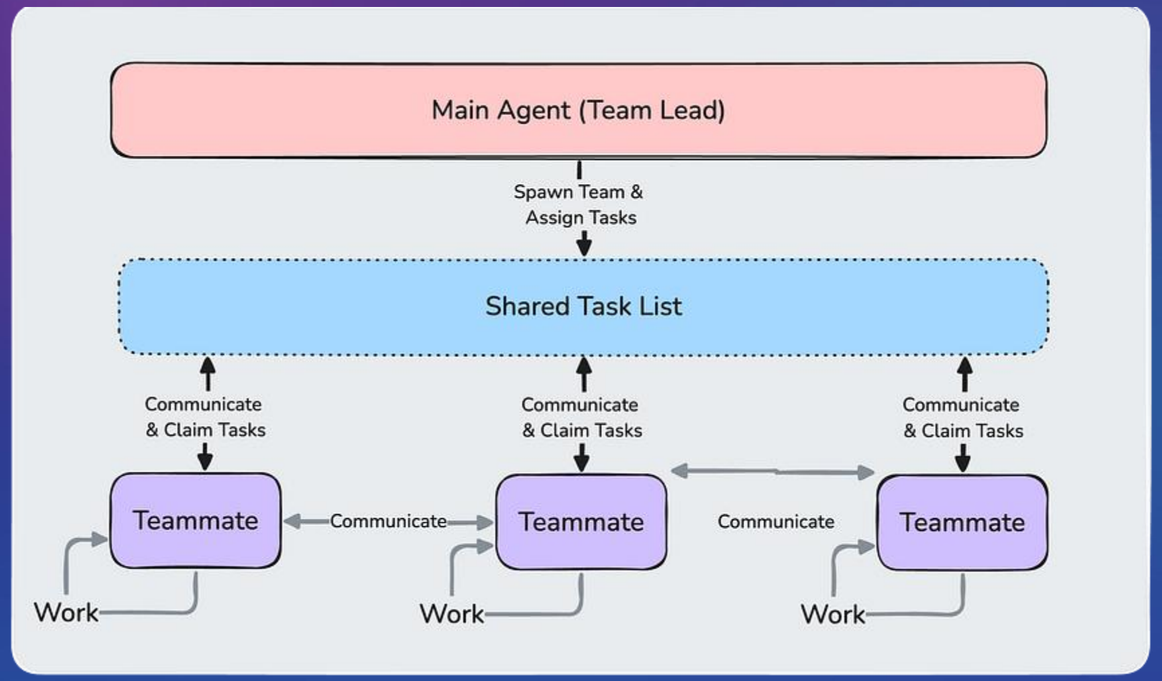
The model didn't change. The prompt did. That's the whole lesson

Claude Agent Teams

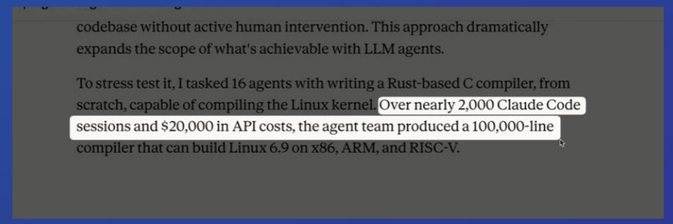
Subagents



Agent Teams



Credits : YouTuber Link


codebase without active human intervention. This approach dramatically expands the scope of what's achievable with LLM agents.

To stress test it, I tasked 16 agents with writing a Rust-based C compiler, from scratch, capable of compiling the Linux kernel. Over nearly 2,000 Claude Code sessions and $20,000 in API costs, the agent team produced a 100,000-line compiler that can build Linux 6.9 on x86, ARM, and RISC-V.

It's Showtime

Three specialist agents. One codebase. Running in parallel

Iron Man

Security Auditor

src/routes/

- SQL injection
- missing auth
- data exposure

Captain America

Auth & Validation

src/middleware/

- Hardcoded secrets
- weak JWT
- no rate limit

Thor

Data Integrity

src/models/

- Plaintext passwords
- default admin
- no pagination

⚡ *All three run simultaneously · Promise.all([ironMan(), captainAmerica(), thor()])*

🎯 **Mission: find every security, auth and data issue in this codebase — before it ships**

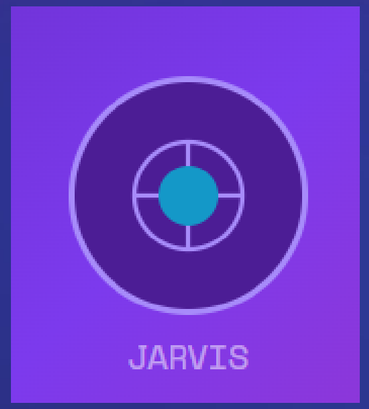
JS COMMUNITY

More at :
https://github.com/FlorianBruniaux/claude-code-ultimate-guide/blob/main/guide/workflows/agent-teams.md

Agents in the JS Ecosystem

Teach It Once, Ship It Right

Build JARVIS, Not Ultron




```
async function runWithCheckpoint(fn) {
  const result = await fn();
  if (result.confidence < 0.75) {
    await requestHumanApproval(result);
  }
  return result;
}
```

Trust boundaries

Write down what this agent is allowed to do without asking — before you build it. Scope permissions tightly. This is the most important design decision you will make.

Checkpoint gates

After each agent completes, require a human approval token before the next starts. Put humans in the loop at the right moments — not every step, but the high-stakes ones.

Audit logs

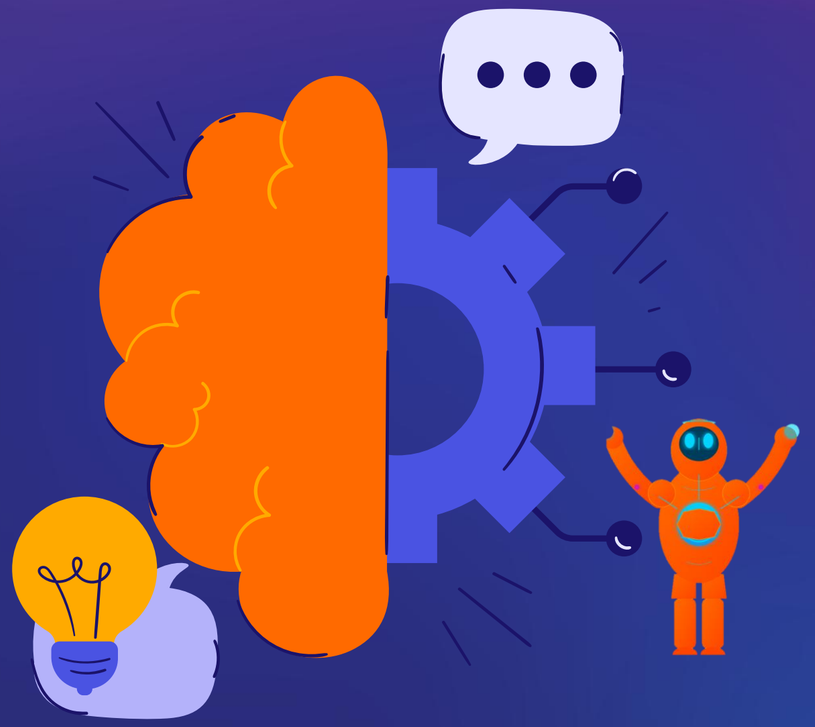
Every tool call, decision, and retry must be logged in a structured, queryable format. You need to reconstruct exactly what the agent did and why — especially when it goes wrong.

Async agents ahead

Agents that run in the background, wake on events, persist memory across sessions, and negotiate with each other. The engineer's new role: architect of the system, not executor of the steps.

JS COMMUNITY

You are the TONY STARK



You define the mission:
Frame the problem, define constraints, set trust boundaries. That is the strategic layer — and it stays human.

JARVIS does the work:
Review code, generate tests, run CI, report to Jira. Agents handle the repetitive high-stakes execution — without you in the loop for every step.

You review, not execute:
Your job shifted from "do the thing" to "approve or redirect the thing." Automated evaluation and human review form a tight loop that improves quality while preserving control.

Monday morning:
Pick one repetitive task, prototype an agent for it. Read the MCP spec. Add structured logging to every AI call you're already making.

JS COMMUNITY

The engineer's role is not disappearing — it is being promoted

Thank You

https://github.com/Ashadeepa/ashadeepa-tech-talks

https://www.linkedin.com/in/ashadeepa-debnath/

JS COMMUNITY

And Welcome to the New Agentic Era!